Balanceo de un robot de dos ruedas con aprendizaje por refuerzo

DQN, Double DQN y A2C sobre un simulador construido con mediciones del robot real

Materia: Aprendizaje por Refuerzo I (2026) — Desafío Nº 1 — MIA, FIUBA
Técnicas: Deep Q-Network, Double DQN, Advantage Actor-Critic (A2C)
Autor: Cesar Yepez
Repositorio: https://github.com/piertotumlocomotor/robot_balance/tree/develop/aprendizaje_refuerzo

---

1. El problema

Un robot balanceador de dos ruedas es un **péndulo invertido**: su equilibrio vertical es inestable y, si nadie actúa, cae en menos de medio segundo. Para sostenerlo, un controlador tiene que decidir 100 veces por segundo cuánto y hacia dónde mover los motores, a partir de la inclinación que mide el sensor (IMU).

El robot existe físicamente (ESP32, dos motorreductores JGB37-520B, dos drivers BTS7960, MPU6050). El objetivo es que un agente **aprenda** esa política sin que se le dé la ecuación del sistema, solo una señal de recompensa, y comparar tres algoritmos vistos en la materia sobre el mismo problema. Como el aprendizaje por prueba y error sobre el robot real lo rompería, se entrena sobre un **simulador construido con los parámetros medidos** del robot.

2. Modelo físico y simulador

2.1 Ecuación del movimiento

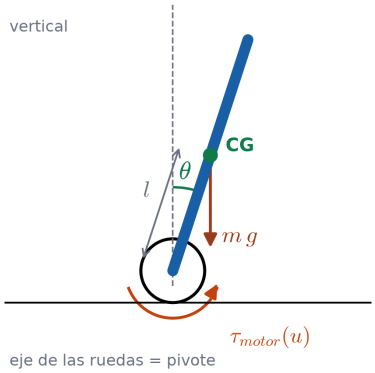


Figura 1 — Diagrama de cuerpo libre. El cuerpo gira alrededor del eje de las ruedas.

Tomando momentos respecto del eje de las ruedas y linealizando para ángulos chicos (sen θ ≈ θ):

$$J\ddot{\theta} = mgl\,\theta + \tau_{motor}(u) \quad \Rightarrow \quad \ddot{\theta} = \omega_0^2\,\theta + K_U\,k(|u|)\,u$$

La aceleración angular tiene dos términos que compiten:

- **La gravedad** (término ω₀²·θ). Empuja a caer, y más cuanto más inclinado está (ω₀² = m·g·l / J). Por eso el equilibrio es inestable.
- **Los motores** (término K_U · k(|u|) · u, que es τ_motor / J). Se lee de derecha a izquierda: **u** es el duty de PWM aplicado a los dos motores (−121 a +121; el signo es el sentido de giro); **k(|u|)** es la fracción de torque disponible por la zona muerta: vale 0 si |u| < 66 (ningún motor vence su fricción interna), 0.57 si 66 ≤ |u| < 73 (solo empuja M1) y 1 si |u| ≥ 73 (empujan los dos); **K_U** convierte duty en aceleración angular (rad/s² por unidad de duty) con los dos motores activos.

2.2 Parámetros

Parámetro	Valor	Origen
ω₀ (frecuencia natural)	8.0 rad/s	medido: video del robot colgado del eje
J (momento de inercia)	0.0159 kg·m²	medido: masa y centro de masa (dos apoyos), J = m·g·l/ω₀²
m·l	0.1040 kg·m	medido: dos apoyos sobre balanza
K_U	**0.043** rad/s² por duty	**supuesto** (ver 2.3)
zona muerta M1 / M2	66 / 73 de duty	medida con el driver anterior, escalada igual que K_U
tope de duty	121 / 255	6 V nominales del motor sobre 12.6 V de batería

2.3 El parámetro que no se pudo medir: K_U

El torque de los motores con el driver nuevo (BTS7960) iba a medirse con balanza, pero el microcontrolador se dañó por un cortocircuito antes del ensayo. K_U se construyó así:

$$K_U = f \cdot \frac{\tau_{M1} + \tau_{M2}}{u_{ensayo} \cdot J} = 1.4 \cdot \frac{0.0443 + 0.0339}{160 \cdot 0.0159} \approx 0.043 \text{ rad/s}^2 \text{ por duty}$$

Término	Valor	De dónde sale
τ_M1, τ_M2	0.0443 y 0.0339 N·m	torque de cada motor medido con balanza, con el driver anterior (L298N), los dos motores encendidos a la vez
u_ensayo	160 de duty	duty al que se midieron esos torques
J	0.0159 kg·m²	inercia del robot actual (2.2): pasa de torque a aceleración angular
f (factor de driver)	**1.4** (supuesto)	cuánto más torque da el driver nuevo al mismo duty

Por qué f está entre 1.0 y 1.6. El torque de un motor frenado es proporcional a la tensión que le llega. El L298N usa transistores bipolares que pierden ≈ 3 V: a duty 160 debía entregar 160/255 · 12.4 V ≈ 7.8 V y el motor recibía ≈ 5 V (estimado por la velocidad de giro). El BTS7960 usa MOSFETs (≈ 10 mΩ) y prácticamente no pierde tensión. Entonces f = 1.0 es el caso pesimista (rinde igual que el viejo) y f ≈ 7.8/5 ≈ 1.6 el optimista (llega toda la tensión). Se adoptó **1.4**, medianamente optimista, dejando margen para pérdidas en cables y batería. La zona muerta medida con el driver viejo (92 y 102 de duty) se divide por el mismo factor, porque también es un umbral de tensión.

$$\theta_{umbral} = \frac{K_U \cdot 121}{\omega_0^2} \approx 4.7^\circ$$

θ_umbral es la inclinación máxima de la que el robot puede recuperarse aun con todo el motor (más allá, la gravedad gana). Es el número que más depende del supuesto y debe validarse en el robot real.

2.4 Simulador y su verificación

Se implementó como entorno de **Gymnasium** (`entorno.py`). La física se integra con Euler semi-implícito en pasos de 1 ms y el agente decide cada 10 ms. Antes de entrenar se hicieron tres verificaciones (`test_entorno.py`), porque un error en el simulador se confunde fácilmente con un agente que aprende mal:

- **(a) Cumple la interfaz de Gymnasium.** La función `check_env` de la biblioteca comprueba que `reset` y `step` devuelven lo que deben, con los tipos y tamaños declarados, y que con la misma semilla se repite el episodio. Garantiza que cualquier algoritmo puede usar el entorno.
- **(b) La física coincide con la solución exacta.** Sin motor, la ecuación tiene solución analítica θ(t) = θ₀·cosh(ω₀·t). Partiendo de 1°, a los 0.3 s el simulador da 5.58° y la fórmula 5.56° (error 0.4%): el integrador y el signo de la gravedad están bien.
- **(c) El problema es resoluble pero no trivial.** Una política al azar cae siempre (en ≈ 35 pasos, 0.35 s) y un controlador PD ajustado a mano sostiene el robot en 50 de 50 episodios. Si el azar sobreviviera, no habría nada que aprender; si el PD cayera, el problema sería imposible o habría un error.

El controlador PD de referencia aplica u = −(K_p·θ + K_d·dθ/dt): un término proporcional a la inclinación (como un resorte que la devuelve a la vertical) y otro a la velocidad con que cambia (como un amortiguador que evita pasarse). Con K_p ≈ 54 de duty por grado y K_d ≈ 260 de duty por rad/s, elegidas a mano, y el resultado redondeado al nivel de duty disponible más cercano. No

aprende nada: sirve de vara para medir a los agentes.

3. Formulación como proceso de decisión de Markov (MDP)

Cada 10 ms el agente mira el estado del robot, elige cuánto motor aplicar, el simulador avanza 10 ms y le devuelve el estado nuevo y una recompensa (Figura 2). Un **episodio** es un intento de balanceo: empieza con el robot apoyado a mano y termina cuando cae o cuando pasan 5 s.

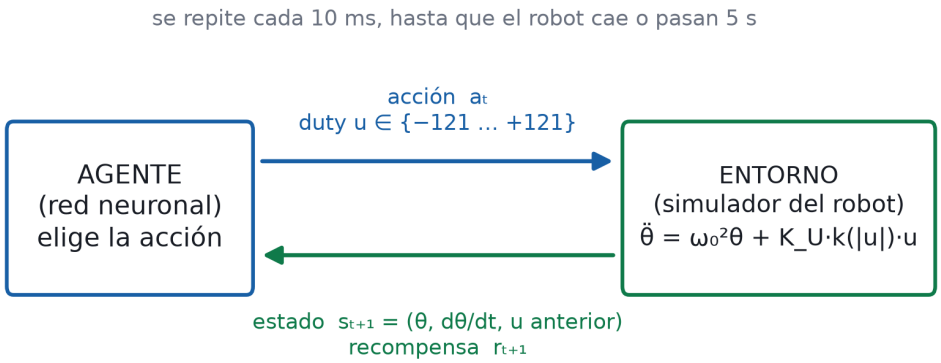


Figura 2 — El bucle agente-entorno en este problema.

Elemento	Definición
Estado s	θ (inclinación, rad), dθ/dt (velocidad angular, rad/s) y u anterior (último duty aplicado)
Acciones a	9 niveles de duty: −121, −105, −90, −75, 0, +75, +90, +105, +121
Recompensa r	$1 - (\theta/10°)^2 - 0.05\cdot(u/121)^2 - 0.05\cdot(\Delta u/242)^2$ por paso; 0 si cae
Fin del episodio	cae: $\lvert\theta\rvert > 10°$ (terminal) · pasan 5 s = 500 pasos (truncado, no terminal)
Estado inicial	θ_0 al azar entre −3.06° y +3.06°; $d\theta/dt_0$ al azar entre −0.15 y +0.15 rad/s
Descuento γ	0.99

Decisiones de diseño

- **Estado normalizado.** Antes de entrar a la red, cada variable se divide por una escala típica: θ por 10°, dθ/dt por $\omega_0 \cdot 10° \approx 1.4$ rad/s y u por 121. Así las tres quedan en el orden de −1 a 1. Sin eso, θ (≈ 0.05 rad) y u (≈ 100) diferirían en más de mil veces, y la red aprendería casi solo de la variable más grande. El u anterior se incluye porque la recompensa castiga cambiarlo (Δu): sin él el agente no podría saber cuánto le cuesta cada acción y el estado no sería markoviano.
- **Acciones discretas.** DQN calcula un valor Q para cada acción y elige el máximo, así que necesita una lista finita de acciones. Se usaron 9 niveles y los mismos para A2C, para que la comparación sea sobre el mismo problema. El menor nivel distinto de cero es 75 porque por debajo de 66–73 el motor no se mueve (zona muerta).
- **Recompensa.** El **1** premia cada paso de pie (sobrevivir es lo principal). **$-(\theta/10°)^2$** premia estar derecho: cuesta poco cerca de la vertical y ≈ 1 cerca de los 10° (anula el premio de seguir de pie). Los términos con **0.05** castigan, con poco peso, gastar motor y conmutarlo bruscamente, que en el robot real calienta el driver y hace vibrar el chasis. Caer da 0 y termina el episodio.
- **Estado inicial = la colocación a mano.** Es el error de apoyar el robot "lo más vertical que el ojo permite": se midió 22 veces, registrando la inclinación cuando el robot estaba quieto (velocidad angular menor a 0.15 rad/s). Dio en promedio 1.98° y como máximo 3.06°. Por eso cada episodio arranca con un ángulo al azar de hasta ±3.06° **y** una velocidad angular al azar de hasta ±0.15 rad/s: son dos variables distintas, de ahí las unidades distintas. Se descartan los pocos estados de los que no se puede salir ni con todo el motor.
- **Descuento γ = 0.99.** El agente maximiza $G = r_1 + \gamma\cdot r_2 + \gamma^2\cdot r_3 + \ldots$: las recompensas no se achican, pero **pesan menos cuanto más lejos están**. Una recompensa a 100 pasos (1 s) pesa $0.99^{100} \approx 0.37$ y una a 500 pasos, 0.007. El agente "mira" aproximadamente el próximo segundo, suficiente porque sin control el robot cae en 0.4 s.

Escala del retorno. Los gráficos muestran el retorno **sin descuento**: la suma de las recompensas del episodio. Como cada paso vale como máximo 1 y hay 500 pasos, el máximo es ≈ 500. Una política al azar cae en ≈ 35 pasos sumando ≈ 0.85 por paso (el término de inclinación crece mientras cae), es decir ≈ 30.

Ejemplo: los primeros pasos de un episodio (DQN entrenada, desde θ = 3°)

t	θ	dθ/dt	acción (duty)	recompensa	Q(s, a elegida)	Q(s, duty 0)
0 ms	3.00°	+0.000 rad/s	-75	0.886	77.3	77.3
10 ms	3.00°	+0.001 rad/s	-121	0.858	77.2	77.1
20 ms	3.00°	-0.017 rad/s	-121	0.861	77.4	77.1
30 ms	2.98°	-0.036 rad/s	-121	0.863	77.5	77.3
40 ms	2.95°	-0.055 rad/s	-121	0.865	77.6	77.5
50 ms	2.92°	-0.074 rad/s	-105	0.880	77.8	77.6

Con el robot inclinado 3° hacia adelante, la red elige empujar en sentido contrario (duty negativo), casi siempre a fondo: en 50 ms la inclinación empieza a bajar y dθ/dt se vuelve negativo, es decir, el robot ya viaja hacia la vertical. Q(s, a) estima el retorno descontado desde ese estado si se toma a y después se juega bien. La diferencia con no hacer nada (duty 0) es chica (0.1 a 0.2), porque no actuar durante un solo paso de 10 ms se puede corregir en el siguiente. Para decidir importa qué acción tiene el mayor Q, no el valor exacto.

4. Agentes

Los tres agentes usan exactamente el mismo entorno; la comparación es entre algoritmos. Las estructuras siguen las implementaciones de la materia (clase 5 y TP3 para DQN, clase 7 para A2C).

4.1 DQN y Double DQN

DQN extiende Q-learning a estados continuos: reemplaza la tabla Q(s, a) por una red neuronal. La red tiene **3 entradas** (las variables del estado), **9 salidas** (un valor Q por acción, así una sola pasada da los nueve y se elige el mayor) y **dos capas ocultas de 64 neuronas** con ReLU, unos 5 000 parámetros. Es el tamaño habitual para problemas de control con pocas variables (el usado en clase para CartPole); no se ajustó. El entrenamiento usa:

- **Replay buffer**: una memoria con las últimas 50 000 transiciones (s, a, r, s', cayó). En cada paso se entrena con 64 transiciones elegidas al azar de esa memoria. Dos motivos: pasos consecutivos son casi iguales y entrenar con ellos en orden sesga a la red; y cada experiencia se reutiliza muchas veces.
- **Red objetivo**: una copia congelada de la red (se actualiza cada 1 000 pasos) para calcular el objetivo y. Si se usara la misma red, el blanco se movería con cada actualización y el aprendizaje oscilaría.
- **Exploración ε-greedy**: con probabilidad ε (de 1 a 0.02 en 50 000 pasos) elige una acción al azar.

$$\text{DQN}: \quad y = r + \gamma\,(1-d)\,\max_{a'} Q_{obj}(s', a')$$

$$\text{Double DQN}: \quad y = r + \gamma\,(1-d)\,Q_{obj}\big(s',\ \operatorname{argmax}_{a'} Q(s', a')\big)$$

La red se ajusta para que Q(s, a) se acerque a y (pérdida de Huber). El máximo de DQN tiende a **sobreestimar** Q, porque elige justamente las estimaciones con error positivo. Double DQN separa los roles: la red principal elige la acción y la red objetivo la evalúa. d vale 1 solo si el robot **cayó**: el corte a los 5 s no es un estado terminal y ahí se sigue estimando el futuro.

```
def objetivo_td(red, red_objetivo, r, s2, d, gamma, double):
    with torch.no_grad():
        if double:
            a2 = red(s2).argmax(dim=1, keepdim=True)        # la red principal elige
            q_sig = red_objetivo(s2).gather(1, a2)          # la objetivo evalúa
        else:
            q_sig = red_objetivo(s2).max(dim=1, keepdim=True)[0]
        return r + gamma * q_sig * (1.0 - d)
```

4.2 A2C (Advantage Actor-Critic)

A2C aprende la política directamente. Un **actor** π(a|s) da la probabilidad de cada acción y un **crítico** V(s) estima el retorno esperado; los dos son redes 3→64→64 (tanh), con 9 salidas el actor y 1 el crítico. Se juegan 16 pasos en 8 entornos en paralelo (en lugar de replay buffer, para no entrenar con datos correlacionados) y se calcula la **ventaja** A: cuánto mejor resultó la acción respecto de lo que el crítico esperaba, con GAE (λ = 0.95). La pérdida combina tres términos:

$$L = -\overline{\log \pi(a|s)\, A} \;+\; 0.5\,\overline{(V(s) - G)^2} \;-\; 0.01\,\overline{H[\pi(\cdot|s)]}$$

El primero sube la probabilidad de las acciones con A > 0; el segundo ajusta al crítico contra el retorno G; el tercero (entropía) evita que la política deje de explorar demasiado pronto. Optimizador RMSprop, tasa de aprendizaje $7 \cdot 10^{-4}$ con decaimiento lineal, 300 000 pasos.

```
adv, ret = ventajas_gae(b_r, v.detach(), v_ult, b_nt, hp.gamma, hp.gae_lambda)

loss_actor = -(dist.log_prob(b_act) * adv).mean()

loss_critico = F.mse_loss(v, ret)

loss = loss_actor + hp.vf_coef * loss_critico - hp.ent_coef * dist.entropy().mean()
```

5. Resultados

Cada agente se entrenó con **10 semillas** (distinta inicialización de la red y distinto azar en la exploración). Las curvas muestran la media móvil de 50 episodios del retorno: la línea es el promedio entre semillas y la banda, ± un desvío estándar.

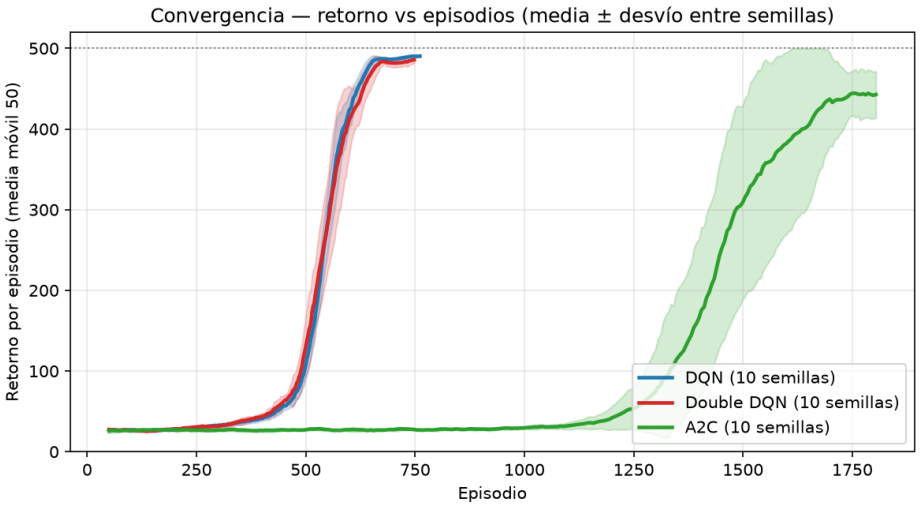


Figura 3 — Convergencia: retorno en función de los episodios.

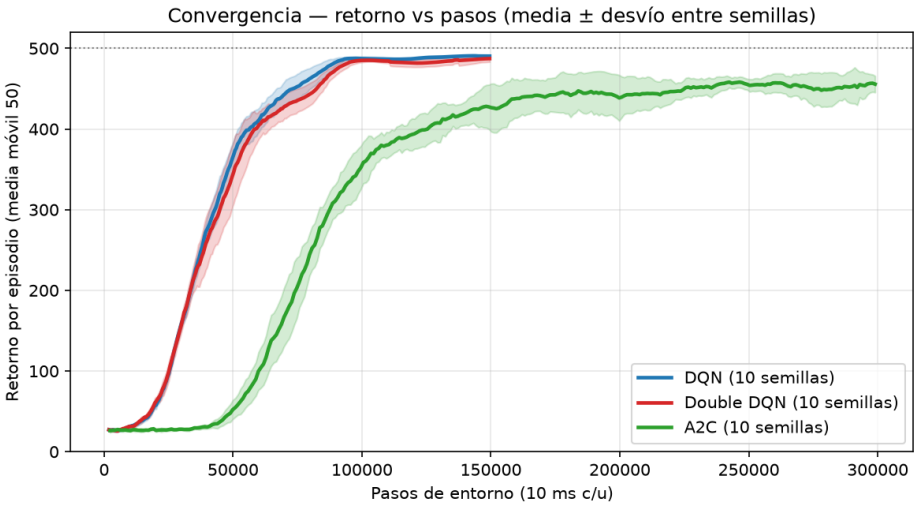


Figura 4 — Convergencia: retorno en función de los pasos de entorno (comparación de eficiencia).

A2C juega muchos más episodios (cortos, en 8 entornos), así que contra episodios aparece desplazado; **contra pasos de entorno** está la comparación justa de cuánta experiencia necesita cada algoritmo.

Evaluación final **sin exploración**, en 200 episodios nuevos por semilla. Media ± desvío entre las 10 semillas; el PD es la referencia que no aprende:

Agente	Retorno	Sobrevive	|θ| medio	|Δduty| medio	Pasos hasta retorno 450
DQN	493.3 ± 3.9	100.0 ± 0.0 %	0.59 ± 0.24°	19.6 ± 20.9	72724 ± 6478
Double DQN	487.6 ± 7.7	100.0 ± 0.0 %	0.80 ± 0.32°	40.8 ± 27.5	80797 ± 6028
A2C	478.3 ± 5.6	99.9 ± 0.2 %	0.31 ± 0.16°	23.0 ± 5.0	166383 ± 28114
PD a mano	494.4	100.0 %	0.66°	32.4	—

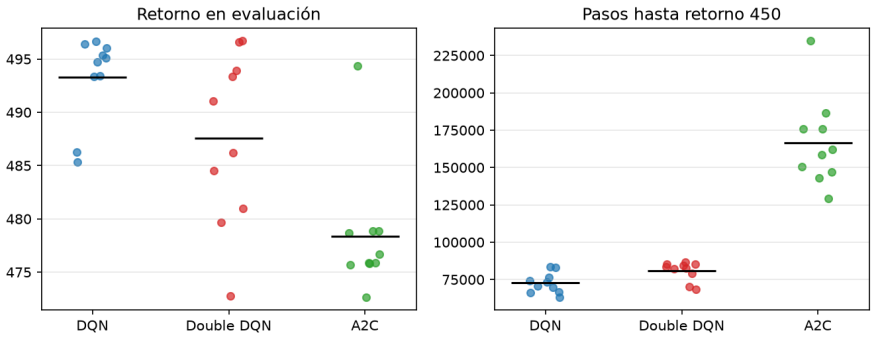


Figura 5 — Un punto por semilla: retorno final y experiencia necesaria. La raya es la media.

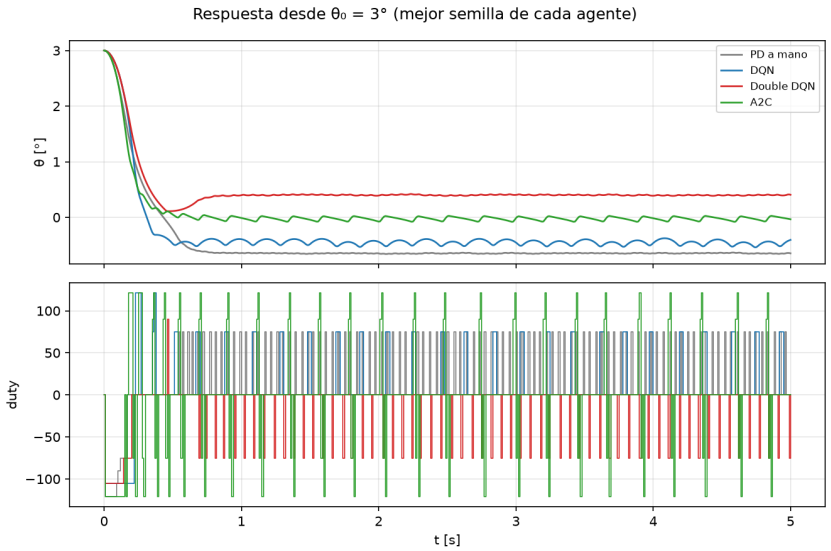


Figura 6 — Respuesta desde θ₀ = 3° (mejor semilla de cada agente): inclinación y duty aplicado.

Discusión

- **Los tres agentes aprenden a balancear el robot** en las 10 semillas. DQN y Double DQN sobreviven el 100 % de los episodios de evaluación; A2C, el 99.9 % (una caída en 2 000 episodios).
- **DQN es el más eficiente en datos**: llega a un retorno de 450 en ≈ 73 000 pasos, Double DQN en ≈ 81 000 y A2C en ≈ 166 000, con una dispersión entre semillas bastante mayor (Figura 5). DQN y Double DQN reutilizan cada transición muchas veces desde el replay buffer; A2C (on-policy) usa cada dato una sola vez.
- **Double DQN no mejora a DQN en este problema**: es algo más lento y obtiene un retorno un poco menor y más disperso (487.6 ± 7.7 contra 493.3 ± 3.9). Con 9 acciones y recompensas acotadas en [0, 1] la sobreestimación del máximo es chica, así que hay poco que corregir; una explicación posible de la leve demora es que su objetivo, más conservador, propaga el valor más despacio.
- **A2C deja al robot más derecho** (|θ| medio 0.31° contra 0.59° de DQN) pero lo hace con pulsos de motor a fondo (Figura 6), que la recompensa castiga; por eso su retorno es algo menor. Su curva de entrenamiento queda por debajo porque mientras entrena la política es aleatoria.
- **El PD ajustado a mano rinde igual que el mejor agente** (494.4 contra 493.3). El modelo es casi lineal y el problema es chico; el aporte del aprendizaje por refuerzo es llegar al mismo desempeño **sin conocer la ecuación**, solo con la recompensa.
- En todos los casos queda una **oscilación residual** de unas décimas de grado cerca de la vertical (Figura 6), por la falta de niveles de motor finos (sección 6).

6. Inconvenientes encontrados y cómo se resolvieron

Un parámetro físico sin medir

El microcontrolador se dañó antes de medir el torque con el driver nuevo. Se reutilizó la medición hecha con el driver anterior, se acotó físicamente cuánto puede mejorar (1.0× a 1.6×) y se adoptó explícitamente 1.4× (sección 2.3), dejándolo como supuesto declarado.

Episodios imposibles de ganar

Con estados iniciales al azar, algunos no se pueden salvar ni con todo el motor, y castigar al agente por eso solo agrega ruido. Se filtran con un criterio analítico: la combinación $\theta + (d\theta/dt)/\omega_0$ (el modo que diverge) tiene que ser menor que θ_umbral.

Fin por tiempo no es caída

Si el corte a los 5 s se tratara como estado terminal, el agente aprendería que al final del episodio "no hay futuro". DQN guarda solo la caída como terminal en el buffer; A2C suma $\gamma \cdot V(s_final)$ a la última recompensa del episodio truncado.

Oscilación residual cerca de 0°

Como el nivel de motor más chico (duty 75) ya supera la zona muerta, no hay empujes finos y el robot queda pulsando cerca de la vertical (Figura 6). Se dejó así porque refleja el actuador real; la mejora natural es agregar niveles más finos o pasar a acción continua.

Variabilidad entre semillas

Con pocas semillas, una diferencia entre algoritmos puede ser solo azar. Se pasó de 3 a 10 semillas, corridas en paralelo, y se reporta media ± desvío.

7. Conclusiones y trabajo futuro

- Se construyó un simulador del robot balanceador a partir de mediciones reales, se formuló el balanceo como MDP y se entrenaron DQN, Double DQN y A2C con 10 semillas cada uno: los tres resuelven el problema.
- DQN fue el mejor compromiso: mayor retorno, menor variabilidad entre semillas y menos de la mitad de experiencia que A2C. Double DQN no aportó mejoras en un problema con pocas acciones y recompensas acotadas.
- El resultado depende de un parámetro supuesto (K_U). Antes de llevar la política al robot hay que medirlo, agregar al modelo el ruido del sensor, la demora del lazo y la dinámica de las ruedas, y probar acciones más finas o continuas para eliminar la oscilación residual.

Referencias

Sutton, R. S. y Barto, A. G. (2018). Reinforcement Learning: An Introduction (2.ª ed.). MIT Press.

Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529–533.

van Hasselt, H., Guez, A. y Silver, D. (2016). Deep Reinforcement Learning with Double Q-learning. AAAI.

Mnih, V. et al. (2016). Asynchronous Methods for Deep Reinforcement Learning. ICML.

Schulman, J. et al. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR.

Towers, M. et al. (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032.

Material de la materia Aprendizaje por Refuerzo I, FIUBA (2026): clases 5 (DQN) y 7 (A2C), TP3.

Código fuente y registro de mediciones: https://github.com/piertotumlocomotor/robot_balance/tree/develop/aprendizaje_refuerzo
Notebook ejecutable en Google Colab:
https://colab.research.google.com/github/piertotumlocomotor/robot_balance/blob/develop/aprendizaje_refuerzo/desafio1_profesor.ipynb